



20

Diving Deeper into Full-Stack Development

After learning how to build and deploy Next.js apps, we are done with our journey into full-stack development with React. In this final chapter, I want to give you an overview and briefly cover various advanced topics that have been left out in this book. This includes concepts such as maintaining large-scale projects, optimizing the bundle size, an overview of **user interface (UI)** libraries, and advanced state management solutions.

In this chapter, we are going to cover the following main topics:

- Overview of other full-stack frameworks
- Overview of UI libraries
- Overview of advanced state management solutions
- Pointers on maintaining large-scale projects

NOTE

As this chapter only gives an overview of advanced topics in full-stack development with links for further reading, there are no code examples, and as such also no technical requirements for this chapter.

Overview of other full-stack frameworks

In this book, we learned about Next.js, the most popular full-stack framework for React. However, other full-stack frameworks might be of interest to you, each coming with its own pros and cons.

Before we can compare the frameworks, though, let's recap the different methods of rendering in React:

- **Client-side rendering (CSR):** Renders components in the browser
- **Server-side rendering (SSR):** Renders components on the server and serves the result
- **Static site generation (SSG):** Renders components on the server and stores them as static HTML, then serves the static HTML
- **Incremental static generation (ISR):** Does SSG on the fly and caches the result for a certain amount of time
- **Deferred site generation (DSG):** Caches all data upon build time and when pages are re-rendered, makes use of that cached data

Additionally, many frameworks (and cloud providers) support the **Edge Runtime**, a subset of standard web APIs that are used to run code on the “edge.” The “edge” in this case refers to serverless compute environments that can be deployed in many locations as close to the customer as possible. For example, if someone accesses your website from Austria, the code will run on the closest server, which may be in Austria or Germany. For someone from the United States, however, the code will run on a server in the United States. This reduces network latency and makes our app load faster.

Now, let's have a look at the different full-stack frameworks.

Next.js

We have already learned about Next.js in this book – it is the most popular full-stack web framework at the time of writing, supporting CSR, SSR, SSG, and ISR. Lately, Next.js defaults to SSG to keep your app as performant as possible, but it still offers the ability to revalidate cached pages and provide SSR when necessary.

Next.js also supports the Edge Runtime, but you have to specifically opt into using it instead of the (default) Node.js runtime. Certain features are also not available on the Edge Runtime.

You can check out Next.js here: <https://nextjs.org/>.

Remix

Remix is a full-stack framework that focuses on web standards.

Unlike Next.js, it does not offer SSG and instead focuses on improving dynamic rendering performance and integration with web infrastructure via SSR. As Remix is fully built on web standards, it does not require Node.js to run, so it can run natively on edge runtimes, such as Cloudflare Workers.

Currently, Remix does not support **React Server Components (RSCs)**, but it has its own patterns that result in the same advantages as using RSCs.

Like Next.js, it supports nested routes (with nested layouts), dynamic routing, and parallel rendering on the server. The Remix router is based on React Router, which makes it easy to understand if you have already worked with React Router. It also supports loading/error states, and a form of Server Actions.

Overall, Remix is a really good alternative to Next.js, especially if you prefer working with standard web APIs and care about edge runtime support. Its goal is to make web development simple again by relying on standards as much as possible.

You can check out Remix here: <https://remix.run/>.

Gatsby

Gatsby mainly focuses on SSG. While it can now also do SSR, the framework authors encourage using SSG as much as possible. Instead of ISR, Gatsby offers DSG, which makes data more consistent on big websites, but at the cost of potentially serving stale data.

Gatsby has recently started offering RSC support and also supports the Edge Runtime. However, like Next.js, it also depends on Node.js APIs and as such only offers a subset of its features for the Edge Runtime.

One advantage of Gatsby is its vast plugin ecosystem, allowing developers to easily integrate new features.

However, one downside of Gatsby is that it does not support nested routes with nested layouts.

While both Next.js and Remix offer support for REST and GraphQL, Gatsby focuses mainly on GraphQL, supporting REST APIs only as a second-class citizen. However, this allows Gatsby to offer plugins that easily integrate various data sources.

Overall, Gatsby can be a great framework if you mainly want SSG, the ability to integrate data from various sources, and an easy-to-learn tool. Instead of throwing all the complexity of a framework at you at once, Gatsby progressively discloses complexity through its plugin ecosystem.

You can check out Gatsby here: <https://www.gatsbyjs.com/>.

Next, we'll provide an overview of a few selected UI libraries.

Overview of UI libraries

In this book, I have purposefully left out UI libraries as they are very opinionated, constantly changing, and would make the code examples significantly longer. In this section, I would like to provide an overview of some selected UI libraries. Feel free to explore them on your own and keep an eye out for other options and new releases in this field!

Material UI (MUI)

MUI is one of the most popular component libraries for React. It supports a wide range of components, including complex components such as data tables. It also has a very extensible theming system, allowing you to adjust it to your style. However, its styling engine, at the time of writing, is incompatible with RSCs, which is something that will be improved in upcoming versions. Use MUI if you generally like its style but want to customize the colors, typography, and spacings to make it your own.

You can check out MUI here: <https://mui.com/>.

Tailwind CSS

Tailwind CSS is a utility-first CSS framework and does not require React. However, it plays well together with React, allowing you to easily style your custom components. As it is CSS-only, you can tailor the React components exactly to your needs. It also means that RSCs are fully supported because Tailwind is simply a set of CSS classes. Use Tailwind if you want to implement a fully custom style for your apps quickly and simply compared to directly using CSS.

You can check out Tailwind CSS here: <https://tailwindcss.com/>.

Tailwind UI

The makers of Tailwind CSS also provide a set of pre-made style-only components that use Tailwind CSS, called Tailwind UI. Check it out if you need inspiration for creating components with Tailwind here:

<https://tailwindui.com/>.

React Aria

React Aria is a simple set of components that have great support for accessibility and internationalization out of the box. By default, the components are style-free, allowing you to build a custom design. You can also use it in combination with Tailwind. Use React Aria if you want to create a design system but do not want to deal with the challenges of creating accessible components.

You can check out React Aria here: <https://react-spectrum.adobe.com/react-aria/>.

NextUI

NextUI is an upcoming UI library using the style of Vercel (the company behind Next.js). It is built on top of Tailwind CSS but offers various components built on top of React Aria, ensuring first-class accessibility support. Like MUI, it also offers many components and is very customizable through themes. Additionally, it supports RSC because it is based on Tailwind CSS. Use NextUI if you like the style and want to customize it a bit, especially if you are developing with a framework that has RSC support.

You can check out NextUI here: <https://nextui.org/>.

Next, we'll provide an overview of advanced state management solutions.

Overview of advanced state management solutions

In this book, we focused on simple state management solutions in React, such as **useState** and contexts. However, in large-scale projects, it might make sense to use advanced state management libraries to deal with complex state. I will give an overview of some selected state management libraries here, but keep in mind that there are many more libraries out there, so feel free to check them out and decide which one fits your project best.

Recoil

Recoil is a state management library for React built by Facebook Open Source. As such, it shares many of the principles of React. It is a very simple but powerful system, where state is stored in atoms, and then derived via selectors. This allows us to, for example, store only the user input of a form in atoms, and a resulting payload in a selector, which derives its state from the atoms.

You can check out Recoil here: <https://recoiljs.org/>.

Jotai

Jotai takes a similar approach to Recoil but simplifies the system by getting rid of selectors and only dealing with atoms. Atoms can then derive state from other atoms. If you want a state management solution that is still simple, but more powerful than **useState**, Jotai is a great solution.

You can check out Jotai here: <https://jotai.org>.

Redux

Redux takes a different approach, offering a central store where all your state is contained, and then only allowing you to change it through actions. This ensures your application behaves consistently, and that the same user actions always result in the same state changes. Redux can be great for applications where actions are essential and when undo/redo functionality is needed (such as certain editors).

You can check out Redux here: <https://redux.js.org>.

MobX

MobX is a signal-based state management library that uses observables to track state changes. When a value is made observable, it can be mutated directly, just like a regular JavaScript variable, but all changes to it trigger observers of the state to execute and re-render components.

You can check out MobX here: <https://mobx.js.org/>.

xstate

xstate is a state machine library, which can be very useful when you have complex user interfaces with various states that need to be kept track of.

You can check out xstate here: <https://state.ai/docs/xstate>.

Zustand

Zustand is a small state management library with a hook-based API that combines values and functions that change the values in stores. Each store then exposes a hook where its values and functions can be used.

You can check out Zustand here:

<https://docs.pmnd.rs/zustand/getting-started/introduction>.

Now, let's wrap up by learning about some pointers on maintaining large-scale projects.

Pointers on maintaining large-scale projects

To keep this book as short and to the point as possible, as well as available to a wide audience, I intentionally left out some topics and technologies. However, these are still very important to get to know when maintaining large-scale projects, so I want to cover them briefly here.

Using TypeScript

TypeScript is JavaScript extended with syntax for types. A type system can be very useful in catching bugs early and give confidence when refactoring a large code base. While it can take some time to get used to typing everything, it becomes a blessing when you realize all problems appear as type errors in your code editor instead of runtime errors for your users.

I would recommend using TypeScript for all new projects. It is easy to learn when you already know JavaScript and integrates well with frameworks such as Next.js.

You can learn more about TypeScript here:

<https://www.typescriptlang.org>.

Setting up a Monorepo

In this book, we were always dealing with a single app at a time. However, large-scale projects often consist of multiple apps, with po-

tentially multiple internal libraries shared between them. For example, you might have two apps, which share UI components in a common UI library. Having all those libraries and apps in separate git repositories can often lead to organizational overhead.

To keep things simple, development teams often decide to set up a Monorepo, which contains all apps and libraries in a single repository. This also makes it easier to keep the code base consistent and keep track of large-scale refactorings.

You can learn more about Monorepos here: <https://monorepo.tools>.

To set up a Monorepo, use a package manager that supports workspaces, such as pnpm (<https://pnpm.io>) or yarn (<https://yarnpkg.com>). Certain tools make creating and maintaining Monorepos easier, such as Turborepo. Check out the guides on <https://turbo.build> to learn how to set up a Monorepo with Turborepo.

Optimizing the bundle size

As your project grows, the JavaScript bundle that's sent to the browser also grows. This can be problematic for devices on slower connections or with slower processors. Sometimes, certain dependencies add a lot to the bundle size, so it is a good idea to regularly check how changes in your project affect the bundle size.

For Vite, you can use **vite-bundle-visualizer** to find out which dependencies are increasing your bundle size:

<https://github.com/KusStar/vite-bundle-visualizer>.

For Next.js, you can use the official **@next/bundle-analyzer** plugin:

<https://nextjs.org/docs/app/building-your-application/optimizing/bundle-analyzer>.

Summary

In this book, we started with the motivation to become a full-stack developer. Then, we set up our development environment and learned about tools that make our lives easier. Next, we got to know Node.js and MongoDB, taking our first steps as backend developers. Then, we implemented a backend for a blog application using Express and Mongoose, and we wrote unit tests for it using Jest. Afterward, we integrated a frontend with our backend using React and TanStack Query, and as such, created our first full-stack web application. Next, we learned how to deploy our application with Docker, and we learned how to set up CI/CD. Then, we added authentication to our application using JWT. We learned how to improve the load time of our app using SSR and developed our own (simple) SSR solution in the process. We then learned how search engines work, and how to make sure customers can find our web application by facilitating SEO and providing metadata for social media embeds. Next, we implemented end-to-end tests using Playwright, making sure that our app always works as expected. Then, we learned how to aggregate and visualize statistics using MongoDB and Victory.

Afterward, we took a detour from REST APIs and developed a backend with a GraphQL API, learning what GraphQL is and what its benefits are. We then developed a frontend that consumes this GraphQL API. Next, we took a break from our blog application and built an event-based chat app using Socket.IO. While doing so, we learned how to create a backend and a frontend, and how to add persistence, in the event-based paradigm. In the last few chapters of this book, we learned about Next.js, a full-stack web development framework. We introduced the app router, a new way to structure your apps, and RSCs, which allowed us to merge the backend and frontend even more, reducing the need for boilerplate code to create APIs, and instead allowing us to directly access code from a data layer inside RSCs. We also learned about advanced concepts and optimizations in

Next.js, such as caching, SEO, and optimized font and image loading. Finally, we learned how to deploy a Next.js app using Vercel, a cloud platform provided by the makers of Next.js, and we created a custom deployment setup using Docker so that we can deploy our app on any other cloud provider (or our own servers).

It has been a long journey. But, as we have seen in this chapter, there are still many more topics to dive deeper into, and the web development ecosystem is changing fast. New technologies keep coming out all the time, especially in terms of RSCs and Server Actions, which, at the time of writing, are still new and upcoming. I expect many more features to be released in this space, so keep an eye out for groundbreaking announcements in the React world!

“Stay hungry. Stay foolish. Never let go of your appetite to go after new ideas, new experiences, and new adventures.” – Steve Jobs